

WORD VECTORING PRO

Technical Documentation

Semantic Structural Matrix Engine · v1.0

`Word vectoring pro.py`

1. Overview

Word Vectoring Pro is a lightweight, self-contained semantic engine that learns word relationships purely from the statistical density of co-occurring tokens inside raw text. It deliberately avoids pre-built corpora, neural weights, or heavyweight databases — instead it grows a live 15-dimensional vector space that converges over time as more text is ingested.

Why was it built? The earlier Autocorrect engine assigned phonetic coordinate vectors to words at learn-time, which meant semantically unrelated words such as `wife` and `knife` ended up neighbouring each other simply because they share a phonetic shape. Conversely, genuinely related words like `engine` and `car` lived in completely separate regions of the space because their spellings differ. Word Vectoring Pro solves both problems by assigning **random initial coordinates** to every new word and then **converging** the vectors of words that repeatedly appear as dominant co-tokens in the same text block.

Core design principle: Random initialisation eliminates phonetic bias. Convergence over many real texts builds genuine semantic proximity. The system is context-driven, not rule-driven.

1.1 Key Capabilities

- Ingests arbitrary plain text and extracts statistically dominant words per block.
- Assigns a 15-dimensional random unit vector to every new token encountered.
- Gradually pulls together the vectors of words that co-dominate the same passages.
- Automatically suspects and flags structural stop-words (words that link to everything).
- Groups the resulting vector space into semantic clusters using cosine similarity.
- Persists all state to JSON so learning accumulates across sessions.

2. File Structure

Parameter / Constant	Value / Description
<code>word_vectoring_pro.py</code>	Main engine — all core logic lives here.
<code>semantic_brain.json</code>	Persisted state: word vectors + known/banned word registry.
<code>relations_history.json</code>	Output snapshot of the last grouper() run — semantic clusters.
<code>testing.py</code>	Automated multi-niche RSS training harness that drives the engine.

3. Global Constants

Parameter / Constant	Value / Description
MEMORY_FILE	"semantic_brain.json" — path for persisted vectors & word registry.
HISTORY_FILE	"relations_history.json" — output path for cluster snapshots.
DIMENSIONS	15 — number of axes in each word's coordinate vector.
LEARNING_SPEED	0.02 — fractional step size used during vector convergence.
MAX_UNIQUE_RELATIONS	100 — unique co-token connections before a word is flagged as a stop-word candidate.

Tuning tip: Raising DIMENSIONS increases the resolution of the semantic space but also increases JSON file size and grouper() computation time. 15 is a practical sweet-spot for single-machine learning runs.

4. Runtime Data Structures

4.1 word_vectors (dict)

Maps every known word to its 15-dimensional coordinate list.

```
{ "engine": [0.1234, -0.8821, 0.3301, ...], # 15 floats
  "car":    [0.1301, -0.8744, 0.3189, ...] }
```

Vectors start random. They drift toward each other each time the two words co-dominate a passage. After many articles, semantically close words cluster in the same region of the space.

4.2 known_words (dict)

Maps every word to either "Acceptable" or "Unacceptable". Unacceptable words are stripped before vectorisation and convergence — they cannot pollute the semantic space.

```
{ "the": "Unacceptable", "engine": "Acceptable", "car": "Acceptable" }
```

4.3 ledger (module-level dict of sets)

Tracks unique co-token pairings seen during the current session. Used exclusively to detect stop-word candidates — it is never persisted to disk.

4.4 suspected_stop_words (module-level dict)

Accumulates words whose ledger entry size has grown past MAX_UNIQUE_RELATIONS during the current run. Cleared after the user reviews and makes decisions at end of session.

5. Public API Reference

5.1 load_brain_memory()

Reads `semantic_brain.json` from disk and returns the two primary data structures. If the file is absent it seeds `known_words` with a small set of hard-coded Acceptable / Unacceptable examples so the engine is never empty on first run.

Returns

Argument	Type	Description
<code>word_vectors</code>	dict	Mapping of word → 15-float list (empty dict on first run).
<code>known_words</code>	dict	Mapping of word → "Acceptable" "Unacceptable".

5.2 save_brain_memory(word_vectors, known_words)

Serialises both data structures into `semantic_brain.json`. Safe to call at any point during a session — subsequent calls overwrite the previous checkpoint.

Parameters

Argument	Type	Description
<code>word_vectors</code>	dict	Current vector map to persist.
<code>known_words</code>	dict	Current word registry to persist.

5.3 vectorization(word, word_vectors)

Creates a brand-new 15-dimensional vector of random floats in the range $[-1.0, 1.0]$ for the given word and inserts it into `word_vectors`. This is the critical departure from the Autocorrect engine: starting positions are fully random so phonetic shape has zero influence on initial placement.

Parameters & Return

Argument	Type	Description
word	str	The new word to register.
word_vectors	dict	The current vector map (mutated in-place).
→ returns	dict	Updated word_vectors with the new entry added.

5.4 converge_vectors(word1, word2, word_vectors)

Nudges word1 and word2 toward each other by a fraction of the gap on each of the 15 axes. The step size is governed by `LEARNING_SPEED` (0.02), meaning a single co-occurrence shifts each coordinate by 2 % of the remaining gap.

Design intent: Small learning steps ensure that a single accidental co-occurrence does not permanently weld two words together. Words must co-dominate many different passages before their vectors converge meaningfully.

Parameters & Return

Argument	Type	Description
word1	str	First word to converge.
word2	str	Second word to converge.
word_vectors	dict	The current vector map (mutated in-place).
→ returns	dict	Updated word_vectors.

5.5 ledger_update(word1, word2)

Registers the pair (word1, word2) in the module-level `ledger` as a set of unique co-token connections. If either word's unique connection count exceeds `MAX_UNIQUE_RELATIONS`, it is added to `suspected_stop_words`. This function has no return value; it operates purely on module globals.

5.6 review_stop_words(word_vectors, known_words)

Presents each stop-word candidate to the user interactively. For each word the user types **y** to permanently blacklist it (removes its vector and marks it Unacceptable) or **n** to keep it in the vocabulary.

Parameters & Return

Argument	Type	Description
word_vectors	dict	Current vector map — may have entries removed.

Argument	Type	Description
known_words	dict	Current word registry — may have entries updated.
→ returns	tuple	(word_vectors, known_words) after user review.

5.7 gobbler(paragraph_text, word_vectors, known_words)

The core ingestion function. Processes one text block end-to-end:

- Lowercases and strips all non-alphanumeric characters.
- Removes any token classified as Unacceptable.
- Registers unseen words with a random vector via vectorization().
- Computes per-token density (occurrences ÷ total clean token count).
- Applies a length-adaptive density threshold to select dominant words:
 - ≤ 150 tokens → 2.5 % threshold (short entries need high repetition)
 - 151–800 tokens → 1.5 % threshold (standard blog-length passages)
 - > 800 tokens → 0.5 % threshold (large essays; even rare words qualify)
- Calls converge_vectors() and ledger_update() for every dominant word pair.

Parameters & Return

Argument	Type	Description
paragraph_text	str	Raw text of any length to ingest.
word_vectors	dict	Current vector map.
known_words	dict	Current word registry.
→ returns	tuple	(word_vectors, known_words) after processing.

5.8 grouper(word_vectors, similarity_threshold=70.0)

The clustering function. Uses cosine similarity to group words whose vectors point in roughly the same direction. For every unvisited word it assembles a pool of neighbours that score ≥ similarity_threshold %. Members of a qualifying pool (size > 1) are recorded with a confidence score equal to their average pairwise cosine similarity within the group.

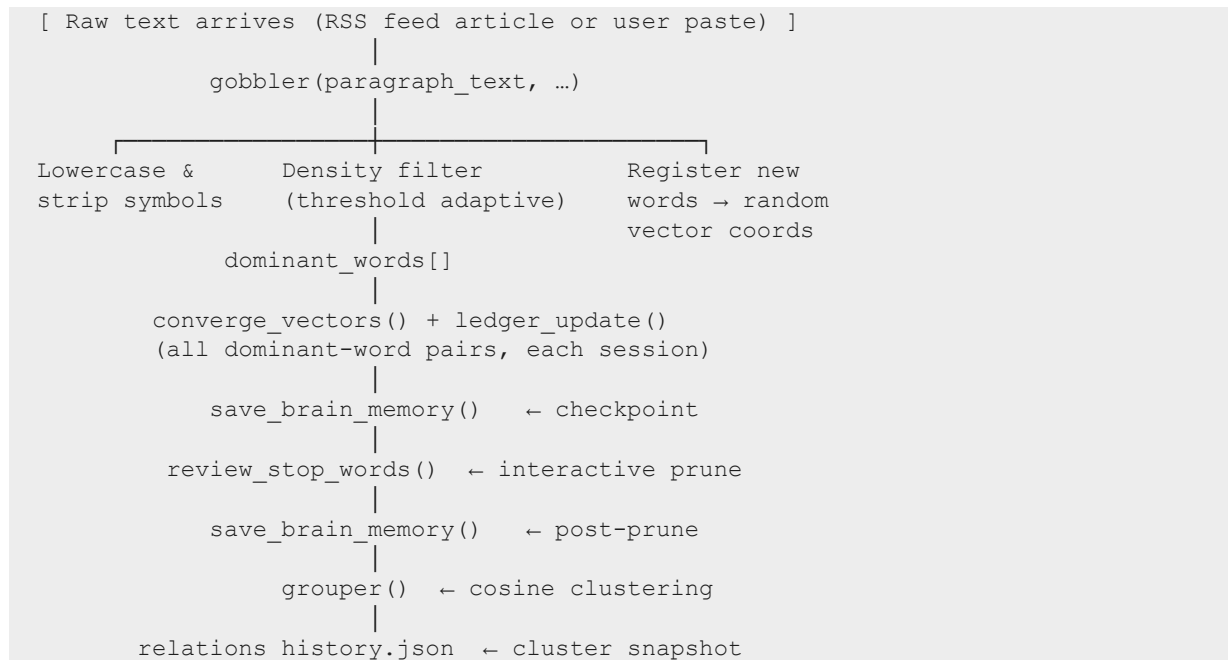
Output format: Returns a dict keyed by group IDs ("r1", "r2", ...). Each value is a dict mapping member words to their intra-group confidence score (0.0–100.0).

Parameters & Return

Argument	Type	Description
word_vectors	dict	The fully trained vector map to cluster.
similarity_threshold	float	Minimum cosine similarity (%) to join a cluster. Default 70.0.
→ returns	dict	Cluster map: { "r1": { "engine": 91.2, "car": 89.7 }, ... }

6. End-to-End Learning Flow

The diagram below shows the sequence of calls made by `testing.py` when running an automated multi-niche training session. The same sequence applies when using the interactive `main()` menu directly.



7. How This Differs from Autocorrect.py

Autocorrect.py	Word Vectoring Pro
Phonetic hash (Soundex) determines initial proximity	Fully random initial vector — no phonetic bias
Fixed-length coordinate vector built at learn-time from character positions	15-D vector converges over multiple real-world contexts

Autocorrect.py	Word Vectoring Pro
wife & knife are neighbours (both map to Soundex W100/K510 region)	wife & knife drift apart unless they genuinely co-dominate the same texts
engine & car never converge without explicit pairing	engine & car converge naturally after a few automotive articles
No stop-word detection — structural words pollute the space	Ledger-based stop-word flagging removes glue text automatically
No clustering — words exist in isolation	grouper() builds named semantic clusters with confidence scores
Designed for single-word spell-checking	Designed for niche-aware semantic reasoning and context building

8. Automated Training Harness — testing.py

testing.py is a standalone driver that feeds Word Vectoring Pro with real-world content from five curated RSS feed niches: Technology, Automotive, Culinary, Finance/Business, and Health/Wellness. It is not part of the core engine — it is the training loop that populates the semantic brain with meaningful, niche-diverse vocabulary.

8.1 How it works

- Calls `load_brain_memory()` to restore any prior session.
- Iterates every RSS URL in `NICHE_BLOGS`, fetches entries with `feedparser`.
- Concatenates title + summary + full content for each article.
- Strips HTML tags via a regex cleaner (`clean_html`) before feeding to `gobbler()`.
- On completion, saves memory, triggers stop-word review, saves again, then runs `grouper()` and writes `relations_history.json`.

8.2 Clean HTML helper

The `clean_html(raw_html)` function uses a compiled regex to strip all HTML tags and decode common named/numeric entities, then collapses whitespace. This prevents tag fragments from being tokenised as vocabulary words and polluting the vector space.

9. Persistent File Schemas

9.1 semantic_brain.json

```
{
  "word_vectors": {
    "engine": [0.1234, -0.8821, 0.3301, 0.5512, -0.2210,
               0.9901, -0.1100, 0.4423, 0.7712, -0.3345,
               0.0012, 0.6634, -0.4421, 0.2213, 0.5541],
    "car": [0.1301, -0.8744, 0.3189, ...]
  },
  "known_words": {
    "engine": "Acceptable",
    "the": "Unacceptable"
  }
}
```

9.2 relations_history.json

```
{
  "r1": { "engine": 91.23, "car": 89.71, "horsepower": 87.44 },
  "r2": { "recipe": 88.10, "ingredient": 85.33, "bake": 82.90 }
}
```

10. Known Limitations & Future Directions

10.1 Current Limitations

- grouper() is $O(n^2)$ in the number of known words — large vocabularies (> 50,000 words) will be slow.
- The ledger and suspected_stop_words dicts are not persisted; stop-word analysis must run within a single session.
- review_stop_words() requires interactive input — it cannot run in headless / fully automated pipelines without modification.
- Convergence is symmetric: pulling word1 toward word2 also pulls word2 toward word1. Highly asymmetric relationships (e.g. hyponyms) are not captured.